

LAPORAN PRAKTIKUM
ALGORITMA & STRUKTUR DATA
MODUL 5



SORTING

Oleh:

Ahmad Riyadhi

NIM. 2510817110004

PROGRAM STUDI TEKNOLOGI INFORMASI
FAKULTAS TEKNIK
UNIVERSITAS LAMBUNG MANGKURAT
MEI 2026

LEMBAR PENGESAHAN
LAPORAN PRAKTIKUM ALGORITMA & STRUKTUR DATA
MODUL 5

Laporan Praktikum Algoritma & Struktur Data Modul 5 : Sorting ini disusun sebagai syarat lulus mata kuliah Praktikum Algoritma & Struktur Data. Laporan Praktikum ini dikerjakan oleh:

Nama Praktikan : Ahmad Riyadhi
NIM : 2510817110004

Menyetujui,
Asisten Praktikum

Mengetahui,
Dosen Penanggung Jawab Praktikum

Fadhil Syahdama Mahatma Putra
NIM. 2410817210026

Andreyan Rizky Baskara, S.Kom., M.Kom
NIP. 199307032019031011

DAFTAR ISI

LEMBAR PENGESAHAN.....	i
DAFTAR ISI	2
DAFTAR TABEL	3
DAFTAR GAMBAR	4
SOAL 1	5
Pembahasan	5
TAUTAN GIT HUB	16

DAFTAR TABEL

Table 1 source code sorting.....	5
----------------------------------	---

DAFTAR GAMBAR

gambar 1 Tampilan awal program	9
gambar 2 Tampilan program jika user memilih opsi 1	10
gambar 3 Tampilan program jika user memilih opsi 2	10
gambar 4 Tampilan program jika user memilih opsi 3	11
gambar 5 Tampilan program jika user memilih opsi 4	11
gambar 6 Tampilan program jika user memilih opsi 5	12
gambar 7 Tampilan program jika user memilih opsi 6	12
gambar 8 Tampilan program jika user memilih opsi 7	13

SOAL 1

Buat Program Sederhana Menggunakan Nama dan Angka NIM Masing-masing :

- Insertion Sort (Nama)
- Merge Sort (Nama)
- Shell Sort (Nama)
- Quick Sort (NIM)
- Bubble Sort (NIM)
- Selection Sort (NIM)



Pembahasan

Table 1 source code sorting

1	#include <iostream>
2	#include <string>
3	#include <conio.h>
4	#include <stdlib.h>
5	#include <algorithm>
6	
7	using namespace std;
8	
9	void insertionSort(string &arr) {
10	int n = arr.length();
11	for (int i = 1; i < n; i++) {
12	char key = arr[i];
13	int j = i - 1;
14	while (j >= 0 && arr[j] > key) {
15	arr[j + 1] = arr[j];
16	j = j - 1;
17	}
18	arr[j + 1] = key;
19	}
20	}

```

21
22 void merge(string &arr, int l, int m, int r) {
23     int n1 = m - 1 + 1;
24     int n2 = r - m;
25
26     string L = arr.substr(l, n1);
27     string R = arr.substr(m + 1, n2);
28
29     int i = 0, j = 0, k = l;
30     while (i < n1 && j < n2) {
31         if (L[i] <= R[j]) { arr[k] = L[i]; i++; }
32         else { arr[k] = R[j]; j++; }
33         k++;
34     }
35     while (i < n1) { arr[k] = L[i]; i++; k++; }
36     while (j < n2) { arr[k] = R[j]; j++; k++; }
37 }
38
39 void mergeSort(string &arr, int l, int r) {
40     if (l < r) {
41         int m = l + (r - l) / 2;
42         mergeSort(arr, l, m);
43         mergeSort(arr, m + 1, r);
44         merge(arr, l, m, r);
45     }
46 }
47
48 void shellSort(string &arr) {
49     int n = arr.length();
50     for (int gap = n / 2; gap > 0; gap /= 2) {
51         for (int i = gap; i < n; i += 1) {
52             char temp = arr[i];
53             int j;
54             for (j = i; j >= gap && arr[j - gap] > temp; j -=
gap)
55                 arr[j] = arr[j - gap];
56             arr[j] = temp;
57         }
58     }
59 }
60
61 void bubbleSort(string &arr) {
62     int n = arr.length();
63     for (int i = 0; i < n - 1; i++) {
64         for (int j = 0; j < n - i - 1; j++) {
65             if (arr[j] > arr[j + 1]) {
66                 swap(arr[j], arr[j + 1]);
67             }
68         }
69     }
70 }
71

```

```

72 void selectionSort(string &arr) {
73     int n = arr.length();
74     for (int i = 0; i < n - 1; i++) {
75         int min_idx = i;
76         for (int j = i + 1; j < n; j++)
77             if (arr[j] < arr[min_idx])
78                 min_idx = j;
79         swap(arr[min_idx], arr[i]);
80     }
81 }
82
83 int partition(string &arr, int low, int high) {
84     char pivot = arr[high];
85     int i = (low - 1);
86     for (int j = low; j <= high - 1; j++) {
87         if (arr[j] < pivot) {
88             i++;
89             swap(arr[i], arr[j]);
90         }
91     }
92     swap(arr[i + 1], arr[high]);
93     return (i + 1);
94 }
95
96 void quickSort(string &arr, int low, int high) {
97     if (low < high) {
98         int pi = partition(arr, low, high);
99         quickSort(arr, low, pi - 1);
100        quickSort(arr, pi + 1, high);
101    }
102 }
103
104 int main() {
105     int pilihan;
106     // Variabel menggunakan Nama dan NIM praktikan
107     string namaAsli = "AhmadRiyadhi";
108     string nimAsli = "2510817110004";
109     string dataNama;
110     string dataNim;
111
112     do {
113         system("cls");
114         dataNama = namaAsli;
115         dataNim = nimAsli;
116
117         cout << "SORTING" << endl;
118         cout << "===== " << endl;
119         cout << "1. Insertion Sort (Nama)" << endl;
120         cout << "2. Merge Sort (Nama)" << endl;
121         cout << "3. Shell Sort (Nama)" << endl;
122         cout << "4. Quick Sort (NIM)" << endl;
123         cout << "5. Bubble Sort (NIM)" << endl;

```



```

124     cout << "6. Selection Sort (NIM)" << endl;
125     cout << "7. Exit" << endl;
126     cout << "Masukkan Pilihan : ";
127     cin >> pilihan;
128
129     cout << "\n--- HASIL EKSEKUSI ---" << endl;
130     switch (pilihan) {
131         case 1:
132             cout << "Data Awal Nama : " << dataNama <<
endl;
133             insertionSort(dataNama);
134             cout << "Hasil Insertion Sort : " << dataNama
<< endl;
135             break;
136         case 2:
137             cout << "Data Awal Nama : " << dataNama <<
endl;
138             mergeSort(dataNama, 0, dataNama.length() -
1);
139             cout << "Hasil Merge Sort : " << dataNama <<
endl;
140             break;
141         case 3:
142             cout << "Data Awal Nama : " << dataNama <<
endl;
143             shellSort(dataNama);
144             cout << "Hasil Shell Sort : " << dataNama <<
endl;
145             break;
146         case 4:
147             cout << "Data Awal NIM : " << dataNim <<
endl;
148             quickSort(dataNim, 0, dataNim.length() - 1);
149             cout << "Hasil Quick Sort : " << dataNim <<
endl;
150             break;
151         case 5:
152             cout << "Data Awal NIM : " << dataNim <<
endl;
153             bubbleSort(dataNim);
154             cout << "Hasil Bubble Sort : " << dataNim <<
endl;
155             break;
156         case 6:
157             cout << "Data Awal NIM : " << dataNim <<
endl;
158             selectionSort(dataNim);
159             cout << "Hasil Selection Sort : " << dataNim
<< endl;
160             break;
161         case 7:
162             cout << "Program Selesai. Terima Kasih." <<

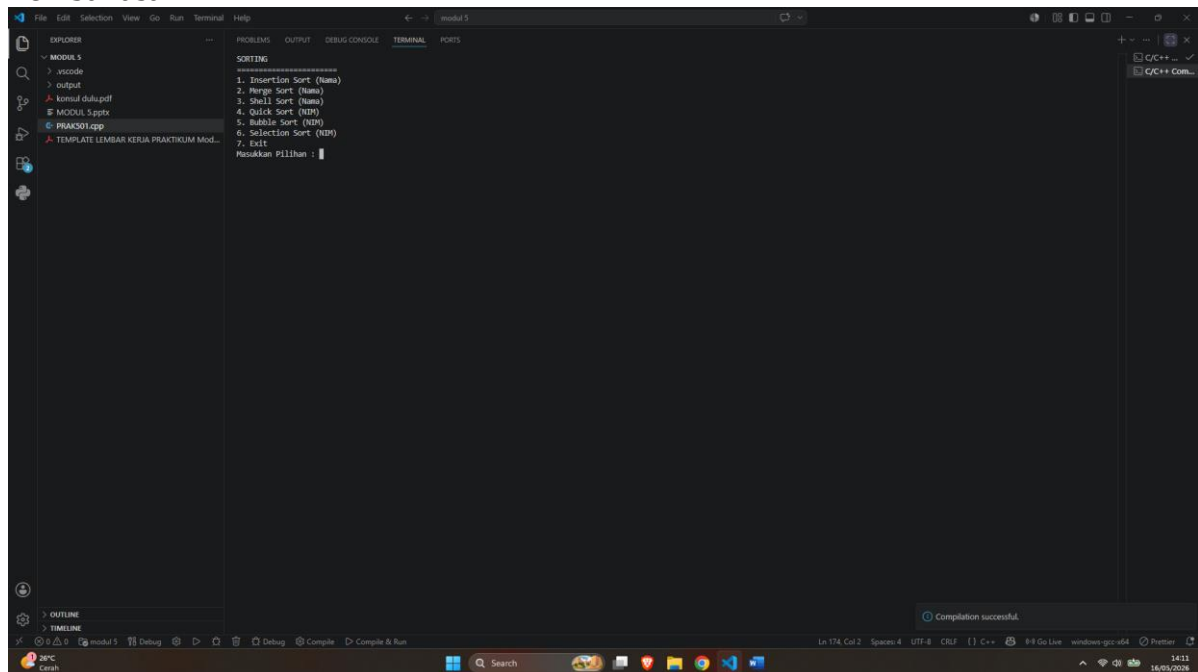
```

```

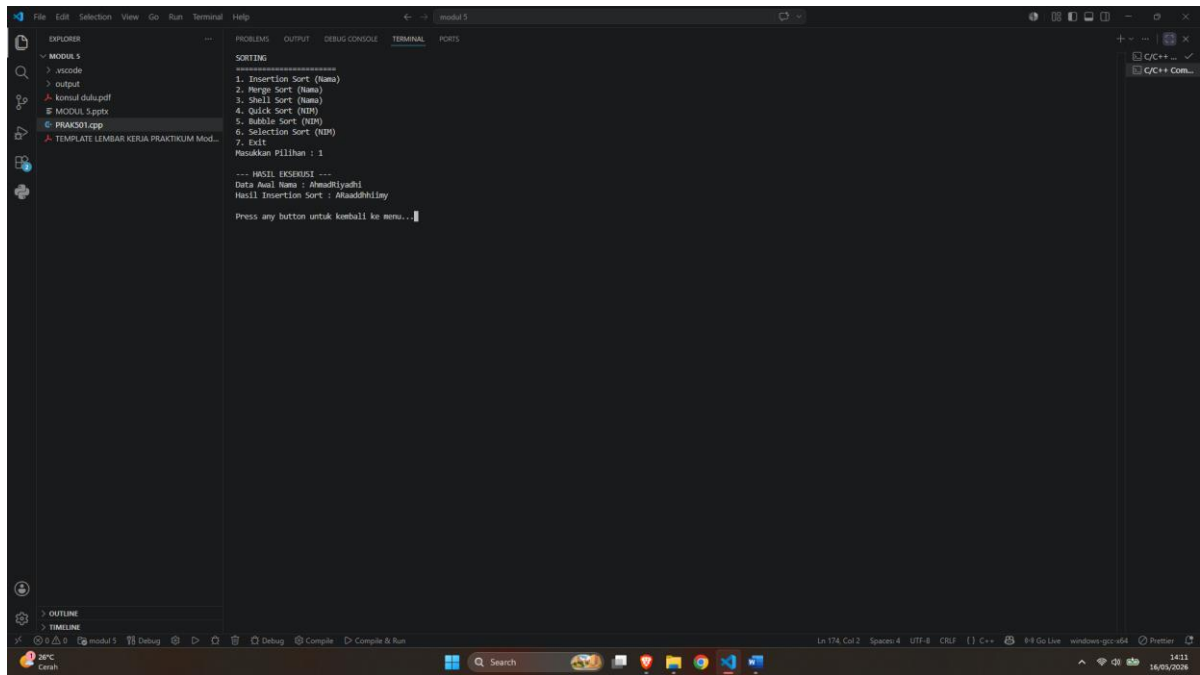
endl;
163         break;
164     default:
165         cout << "Pilihan tidak valid!" << endl;
166     }
167     if(pilihan != 7) {
168         cout << "\nPress any button untuk kembali ke
menu...";
169         getch();
170     }
171     } while (pilihan != 7);
172     return 0;
173 }
174 }

```

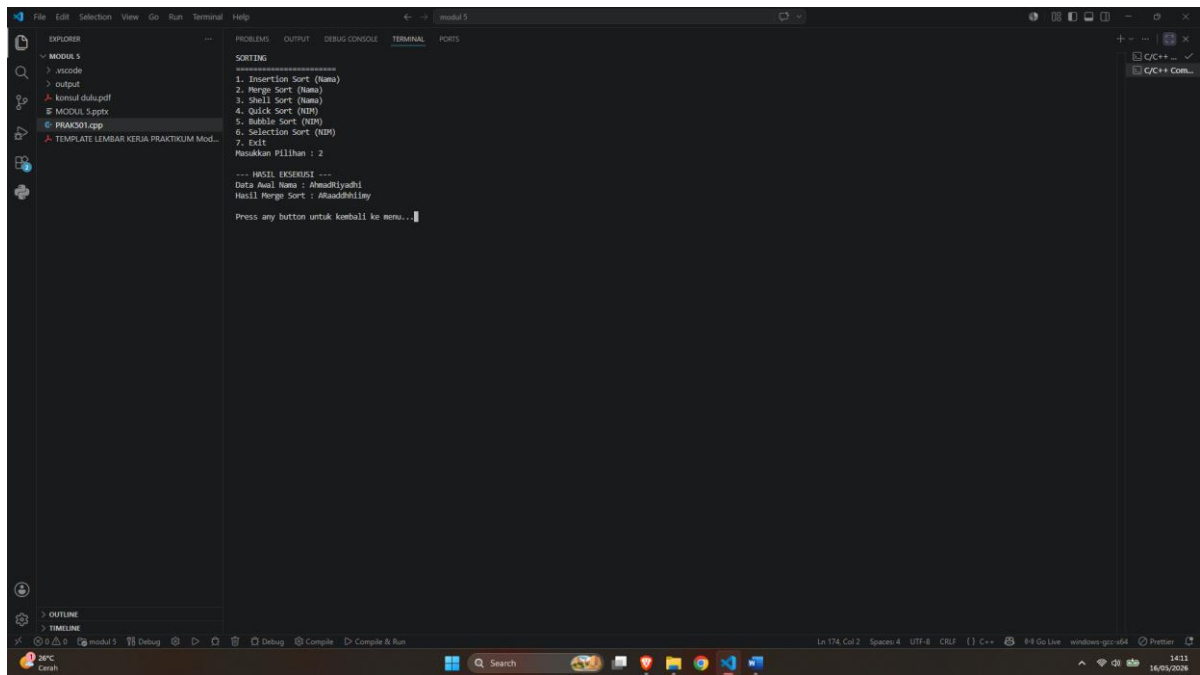
Pembahasan



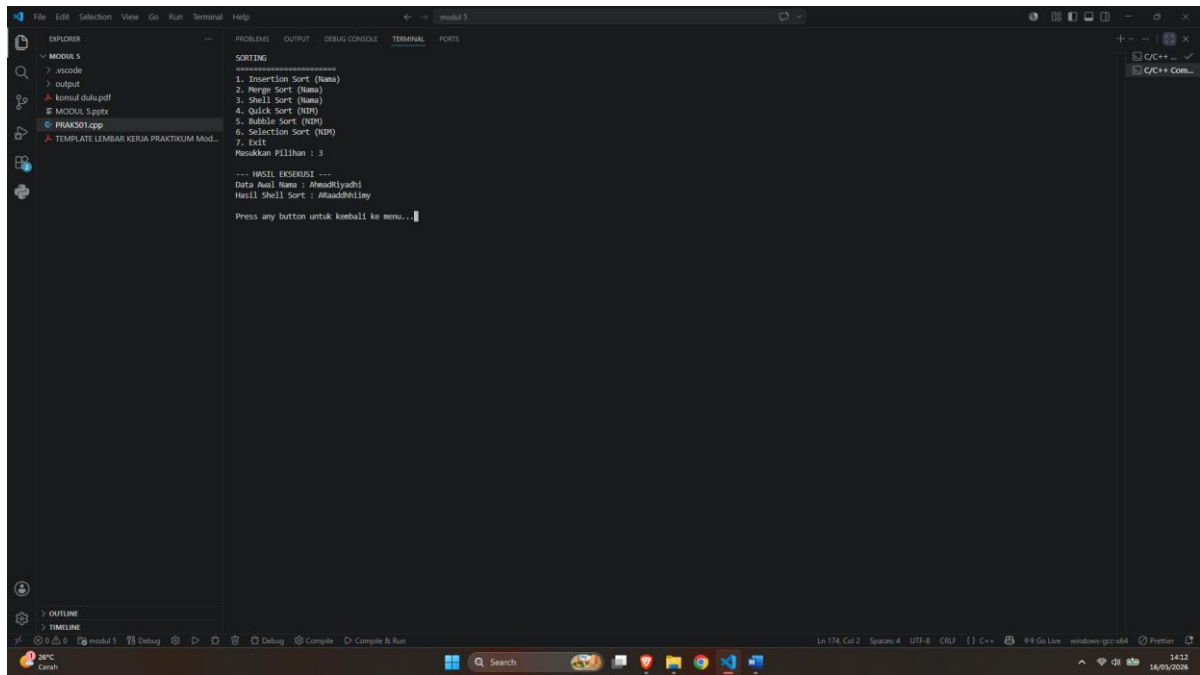
gambar 1 Tampilan awal program



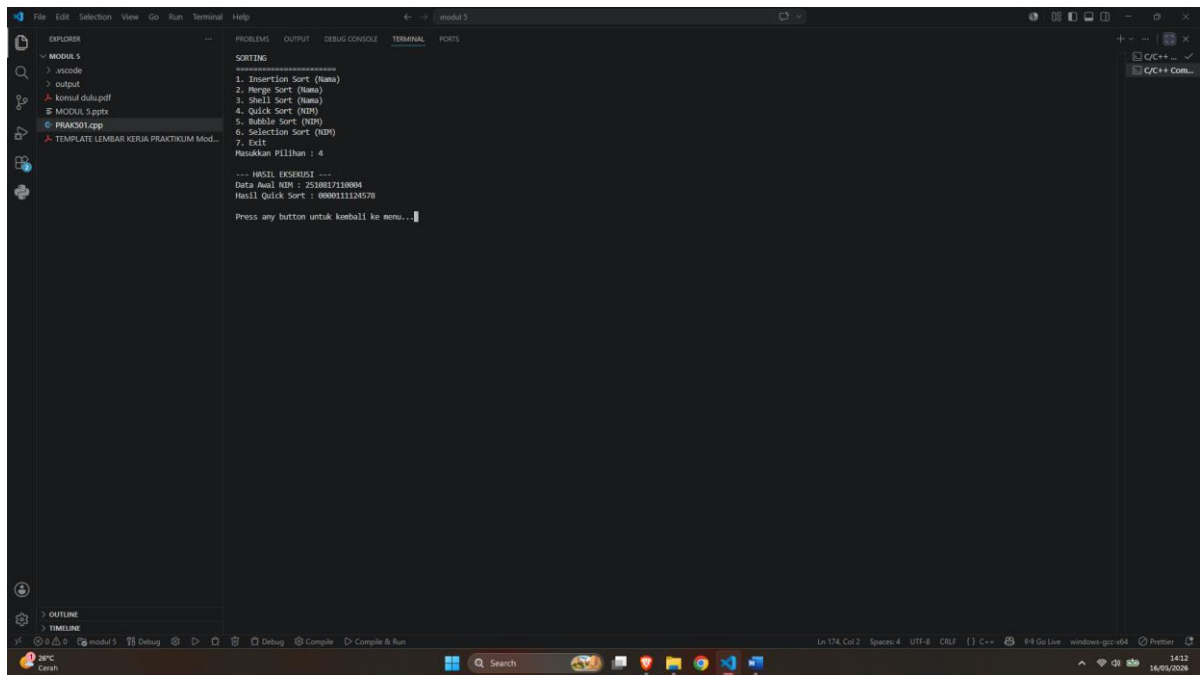
gambar 2 Tampilan program jika user memilih opsi 1



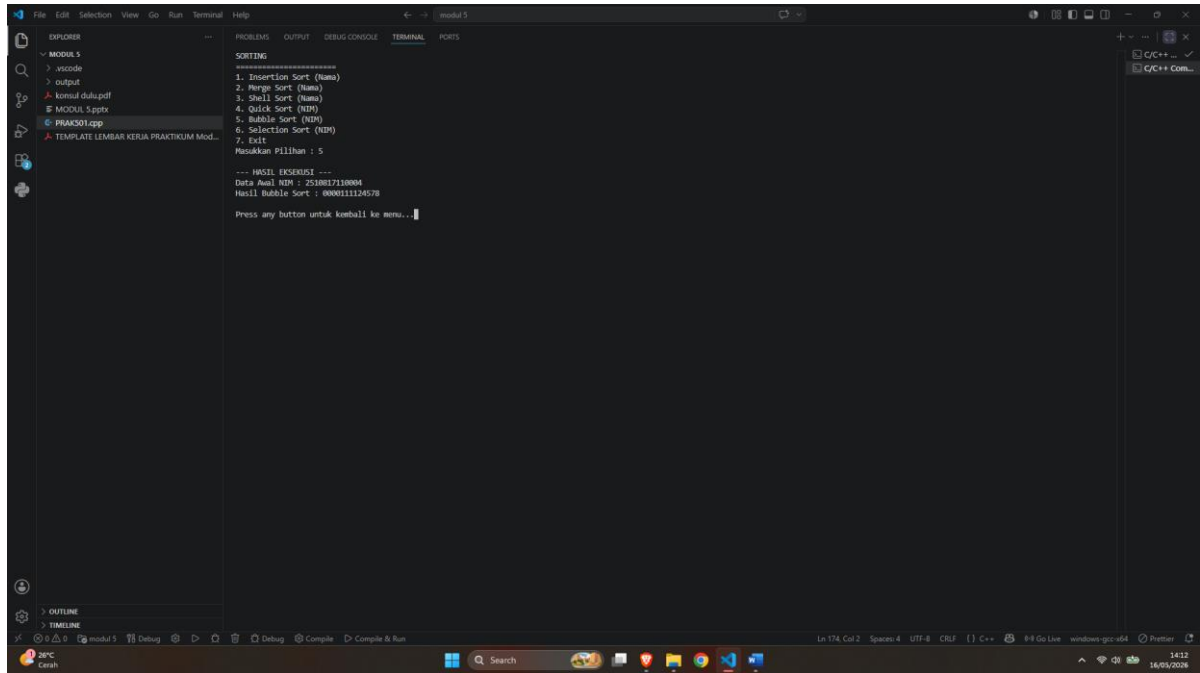
gambar 3 Tampilan program jika user memilih opsi 2



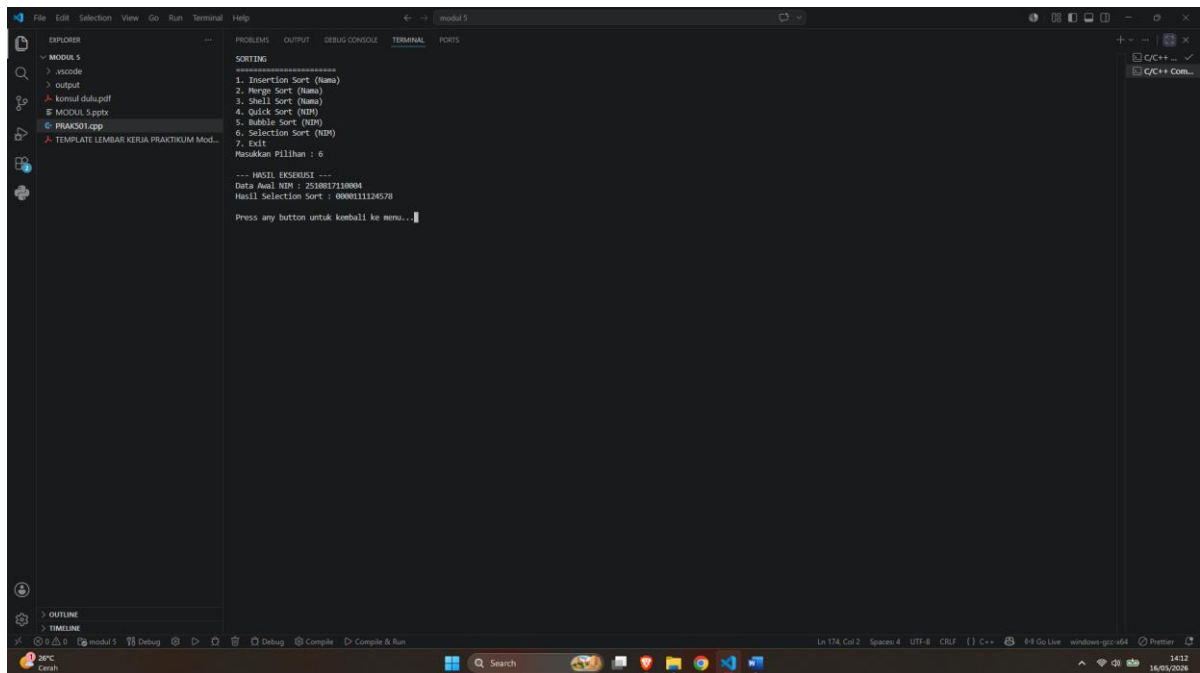
gambar 4 Tampilan program jika user memilih opsi 3



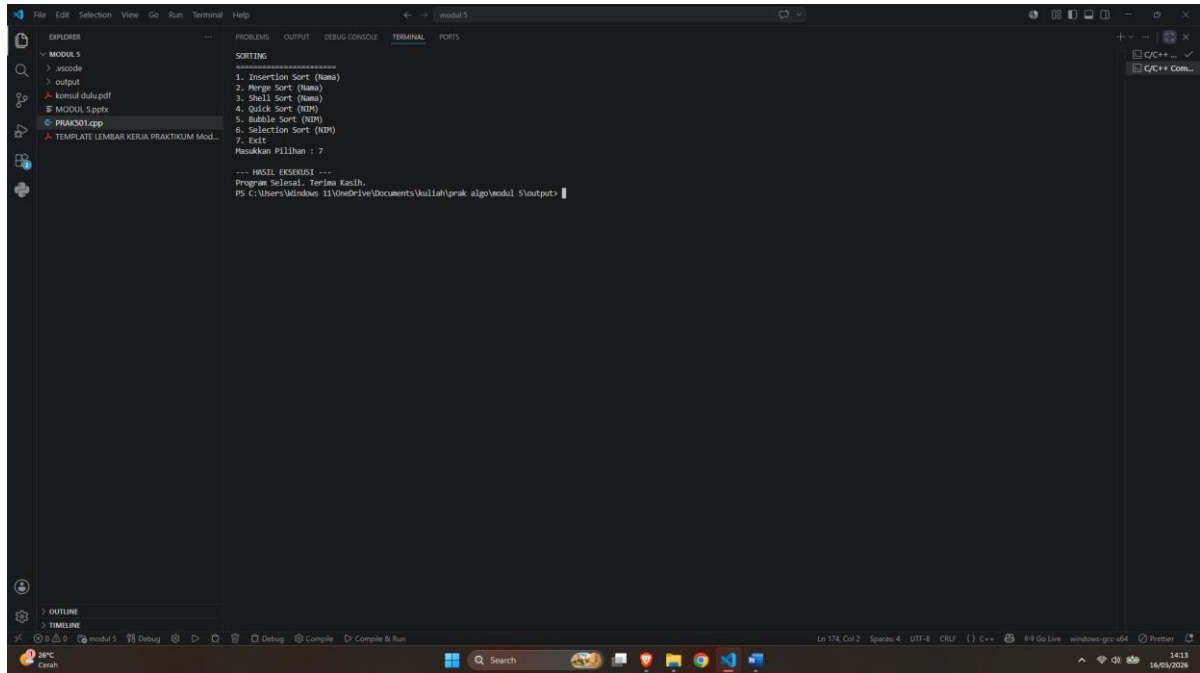
gambar 5 Tampilan program jika user memilih opsi 4



gambar 6 Tampilan program jika user memilih opsi 5



gambar 7 Tampilan program jika user memilih opsi 6



gambar 8 Tampilan program jika user memilih opsi 7

PEMBAHASAN

Berikut adalah lampiran teks lengkap untuk bagian **C. Pembahasan** yang disesuaikan secara mendalam berdasarkan struktur baris kode program terbaru:

C. Pembahasan

□ **Baris 1 - 7:** Sintaks ini adalah bagian persiapan library agar program bisa berjalan. `iostream` digunakan untuk fungsi input/output (`cin` dan `cout`). `string` digunakan untuk mengolah tipe data teks. `conio.h` digunakan untuk fungsi `getch()` yang menahan layar terminal. `stdlib.h` digunakan untuk memanggil perintah sistem seperti `system("cls")` untuk membersihkan layar. `algorithm` digunakan untuk menyediakan fungsi `swap()` yang menukar posisi dua data. Perintah `using namespace std;` digunakan agar kita tidak perlu mengetikkan `std::` pada setiap fungsi dasar.

□ **Baris 9 - 20:** Fungsi `insertionSort` mengurutkan karakter dalam string dengan metode Insertion Sort. Program mengambil satu karakter pada indeks tertentu dan menyimpannya di variabel `key`. Kemudian, program membandingkan `key` dengan karakter-karakter di sebelah kirinya secara mundur. Jika karakter di sebelah kiri nilainya lebih besar dari `key`, maka karakter tersebut digeser satu langkah ke kanan. Setelah pergeseran selesai, nilai `key` diletakkan di posisi kosong yang sudah tepat. Parameter `&arr` digunakan agar string asli langsung berubah.

□ **Baris 21 - 46:** Fungsi merge dan mergeSort menjalankan metode Merge Sort. Pertama, mergeSort membagi teks menjadi dua bagian secara terus-menerus hingga tersisa satuan karakter tunggal. Setelah terbagi, fungsi merge bertugas menggabungkan kembali potongan-potongan tersebut secara berurutan. Fungsi merge menggunakan perintah substr() untuk menyalin sementara bagian kiri (L) dan kanan (R), kemudian membandingkan keduanya untuk disusun kembali ke dalam string utama dari nilai terkecil hingga terbesar.

□ **Baris 48 - 59:** Fungsi shellSort merupakan versi optimasi dari Insertion Sort. Program tidak langsung membandingkan karakter yang bersebelahan, melainkan membandingkan karakter yang jaraknya dibatasi oleh nilai gap. Awalnya jarak gap diatur setengah dari total karakter, lalu perlahan dibagi dua pada setiap putaran hingga jaraknya menjadi 1. Cara ini sangat efektif untuk memindahkan karakter yang letaknya jauh dengan jumlah pergeseran yang lebih sedikit.

□ **Baris 61 - 70:** Fungsi bubbleSort mengurutkan karakter menggunakan metode Bubble Sort. Program berjalan menggunakan dua lapis perulangan untuk membandingkan dua karakter yang letaknya persis bersebelahan. Jika karakter di sebelah kiri lebih besar dari karakter di sebelah kanan, program langsung menukar posisi keduanya menggunakan perintah swap(). Proses perbandingan ini diulang berulang kali dari awal hingga ke akhir baris sampai seluruh karakter tersusunurut.

□ **Baris 72 - 81:** Fungsi selectionSort mengurutkan data dengan metode Selection Sort. Program memindai (mencari) seluruh bagian data yang belum terurut untuk menemukan karakter dengan nilai paling kecil. Indeks dari karakter terkecil tersebut disimpan ke dalam variabel min_idx. Setelah pencarian satu putaran selesai, karakter terkecil itu langsung ditukar (swap()) letaknya dengan karakter paling depan dari area yang sedang diperiksa.

□ **Baris 83 - 102:** Fungsi partition dan quickSort menjalankan metode Quick Sort. Program memilih satu karakter sebagai titik acuan atau pivot (dalam hal ini menggunakan karakter paling akhir). Fungsi partition kemudian memilah data dengan memindahkan semua karakter yang nilainya lebih kecil dari pivot ke sebelah kiri, dan karakter yang lebih besar ke sebelah kanan. Proses pemisahan ini diulangi terus untuk kelompok sebelah kiri dan kanan sampai seluruh data terurut sempurna.

□ **Baris 104 - 173:** Sintaks int main() adalah blok program utama yang pertama kali dijalankan. Di sini disiapkan variabel namaAsli dan nimAsli yang berisi data praktikan.

Program menggunakan perulangan do-while untuk menampilkan menu operasi secara terus-menerus selama users tidak memasukkan angka 7. Di setiap awal putaran, variabel string dataNama dan dataNim di-reset kembali ke nilai aslinya agar selalu diproses dari keadaan acak. Pilihan users dieksekusi melalui struktur switch-case. Pada akhir putaran, getch() menahan layar agar hasil bisa dibaca, lalu layar dibersihkan dengan system("cls").

TAUTAN GIT HUB

<https://github.com/TebangLiar/Praktikum-AlgoritmaDanStrukturData-P1/tree/main/Modul%205>